DS 5110: Project Iteration #2

Lon Pierson & Armand Meskin

Written: Fall 2025

Dataset Used

Source: `https://www.kaggle.com/datasets/arashnic/faces-age-detection-dataset`

Our dataset includes 23,158 total images of human faces: 3,252 images labeled with specific ages (ranging from 1 to 116 years old) in their filenames, and 19,906 images categorized into three age groups (young, middle, old). All images are in JPG format with varying resolutions. This publicly available dataset provides a robust foundation for training our age classification model.

Project Scope

Our project will create an integrated system combining machine learning and database management to efficiently classify and store human facial images by age category. The scope is bounded by several key decisions: we will focus on single-face classification (not multi-face detection), command-line operation (not web/mobile interfaces), and age category classification (not exact age prediction). We will store these photos along with relevant age information in a SQL Database that we create. We will use the provided data from this set as the basis for our project, but in terms of scope, the end product will be one that can store the faces and age information for any number of photographs, including ones that didn't have age labels provided initially.

The project encompasses data preprocessing and cleaning, CNN model development using deep learning frameworks, SQL database schema design with appropriate normalization, and Python scripts for automated image ingestion and classification. We explicitly exclude real-time video processing, facial recognition/identity matching, and web application development to maintain realistic project boundaries for the semester timeline.

Objectives

The main objective of our project is to efficiently store and classify human faces into different age bins in a queryable SQL Database. Users should be able to easily add a photo of a human face to the database, which our CNN model will quickly categorize into one of several age groupings. Being able to quickly query the number of people of certain age demographics appearing in a dataset is a powerful tool that could be used by companies to gain better insight into sales demographics, marketing effectiveness, or customer base composition.

Specifically, we aim to develop a Convolutional Neural Network capable of classifying faces into age bins with at least 75% accuracy, design and implement a normalized SQL database schema that efficiently stores image data and classification results, create a pipeline that allows seamless addition of new images with automatic classification, and enable fast querying of age demographic information. For example, a retail company analyzing customer photos could determine demographic patterns, or a social media platform could better understand their user base age distribution.

Team Member Contributions

Armand Meskin - Database Architecture

Armand will focus on creating the SQL Database and ensuring that the design is modular, intuitive, and efficient. His responsibilities include designing the database schema with proper normalization, implementing referential integrity and indexing strategies for query performance, creating efficient queries for demographic analysis, and developing the Python-SQL integration layer using appropriate database connectors. Armand brings strong SQL fundamentals and database design principles to the project, and will develop skills in advanced query optimization and handling binary image data in databases throughout the project.

Lon Pierson - Machine Learning Development

Lon will work on cleaning up the dataset for training our CNN model for the purposes outlined in the Objectives section. He will also create and train the CNN to ensure that we have a model accurate enough to hold up to the critical lens of potential stakeholders. His responsibilities include data preprocessing and augmentation to improve model generalization, CNN architecture design and hyperparameter tuning, model training and validation with performance evaluation, and creating the image classification pipeline that interfaces with the database. Lon brings strong Python programming and machine learning fundamentals, and will develop deep learning framework proficiency and CNN architecture design skills during the project.

Collaboration and Skills Development

Both team members have identified areas for growth in their specialized domains. To address skills gaps, we will complete relevant TensorFlow and SQL optimization tutorials during the initial project phase, conduct peer knowledge-sharing sessions where each member teaches the other about their domain, consult official documentation and online resources when encountering implementation challenges, and schedule regular code reviews to ensure quality and knowledge transfer between database and machine learning components.

Tools and Technologies

We will utilize a comprehensive set of tools throughout this project. For version control and collaboration, we'll be using GitHub to store all aspects of the project (aside from the training data itself as it's too large to include in the repo, and storing training data in repositories is generally regarded as poor practice). The repository will maintain separate branches for database development and model development, merging to main only after testing. For the coding aspects of our project, we'll be using Visual Studio Code since it's easily adaptable to any different language we could need to use for our project and provides excellent extensions for both Python and SQL development.

For machine learning and data processing, we will use TensorFlow/Keras as our deep learning framework for building and training our CNN model. This is a critical distinction from our initial planning - we specifically need TensorFlow or Keras because SKLearn does not support Convolutional Neural Networks. We'll be making use of NumPy for numerical computing and array operations, the Pandas package to assist with data cleaning and CSV handling for metadata management, and OpenCV or Pillow for image preprocessing, resizing, and format conversion. For visualization of model training progress and performance metrics, we'll use Matplotlib and Seaborn. SKLearn will still be utilized, but only for evaluation metrics and data splitting utilities rather than the CNN itself.

For database management, we have selected SQLite as our SQL database system. SQLite is a lightweight database engine that is perfect for our project scale and provides easy integration with Python through the native sqlite3 library. This choice allows us to focus on database design and query optimization without the overhead of server configuration that would come with MySQL or PostgreSQL.

Programming Languages Used

We will be using Python exclusively for this project, as Python is what both of us are most comfortable using and it has an extensive list of libraries to pull from that can easily and effectively assist with all aspects of our project. As outlined in the tools section of this report, there are many libraries such as TensorFlow/Keras, Pandas, NumPy, OpenCV, and sqlite3 that make using Python for this project an easy decision. The language provides seamless integration between machine learning pipelines and SQL databases, excellent support for image processing and deep learning, and a mature ecosystem of data science tools. All these external libraries, along with clean integration with SQL Databases through sqlite3, make Python the obvious choice for us to create our project with. Python's versatility allows both team members to work in a single language environment while accessing specialized functionality through its rich library ecosystem.

Development Environment and Progress Tracking

Our development environment is built around Python 3.10+ with virtual environments to manage dependencies. The GitHub repository follows a clear organizational structure with separate directories for data processing scripts, model code, database schema files, utility scripts, documentation, and tests. A requirements.txt file will track all Python dependencies to ensure reproducibility across team member environments.

We have created an Excel progress tracker that monitors task completion throughout the project. The tracker includes task breakdown with assigned team members, estimated and actual completion times, dependencies between tasks, status tracking (Not Started, In Progress, Testing, Complete), and a notes section for documenting blockers or changes. This tracker is updated weekly and is included in our repository under the docs directory. The tracker currently shows completion of initial project planning tasks and setup of development environments, with upcoming tasks for dataset preprocessing and initial model architecture design.

Submission Verification

We confirm that this submission meets all requirements for Iteration #2. This three-page project introduction summarizes our project scope (age-based facial classification with SQL storage), objectives (75%+ accuracy CNN model with efficient database querying), team members' contributions (Armand: database architecture, Lon: ML development), tools (GitHub, VS Code, TensorFlow/Keras, SQLite, NumPy, Pandas, OpenCV), and programming languages used (Python exclusively). The dataset link has been provided above for the publicly available Kaggle faces dataset. Progress has been tracked using an Excel file that is included in our GitHub repository submission. The GitHub repository has been initialized with proper structure, all team members have repository access and configured development environments, and this PDF report has been created in Overleaf and is ready for stakeholder review.

Links

GitHub Repository:
https://github.com/piersonl/Age-Recognition
Progress Tracker:
https://docs.google.com/spreadsheets/d/1OkgbXiSMewRiRqhabmr40UJ2gUDZ9TSyViRm1Y2h7cs/edit?usp=sharing